

UNIVERSIDAD NACIONAL DE CÓRDOBA

Facultad de Ciencias Exactas, Físicas y Naturales

Ingeniería en Computación

Trabajo Práctico N.º 5

Transmisión de Datos: de ICMP y ARP a Sockets TCP entre Dos Computadoras

Asignatura: Redes de Computadoras

Grupo: MiLANesas

Alumnos: Tomas Brigido Kevin Cufre
Rocco Delgado Facundo Lugones
Javier Siñuka Agustin Tosolini

Fecha: Septiembre de 2026

Índice

1. Ítem 1: ICMP y primer contacto con Wireshark	1
1.1. Fundamentos de ICMP	1
1.2. Configuración y entorno de medición	1
1.3. Procedimiento y resultados generales	2
1.4. Identificación de las capas y sus campos	4
1.5. a) MAC destino y alcance del direccionamiento	4
1.6. b) Comparación entre Echo Request y Echo Reply	5
1.7. c) Ubicación, tamaño y contenido del payload	7
1.8. d) Análisis del TTL	8
1.9. e) Encapsulación y distribución de bytes	9
1.10. Síntesis y evidencias de la práctica	10
2. Ítem 2: ARP: de una IP a una dirección MAC	11
2.1. Fundamentos de ARP y resolución de direcciones	11
2.2. Inspección de la caché ARP local	13
2.3. Generación de tráfico ARP y captura con Wireshark	14
2.4. Análisis comparativo de ARP Request y ARP Reply	15
2.5. Síntesis y evidencias de la práctica	18
3. Ítem 3: TCP y UDP “a mano” con ncat	19
3.1. Fundamentos de TCP, UDP y puertos	19
3.2. Configuración y procedimiento experimental	19
3.3. Mensajes transmitidos y resultados generales	20
3.4. a) Tráfico anterior al primer mensaje	21
3.5. b) Datagramas por mensaje y confirmaciones	22
3.6. c) Comparación de los encabezados	23
3.7. d) Cierre del cliente	24
3.8. e) Cantidad de paquetes para enviar la misma frase	25
3.9. f) Comportamiento ante puertos cerrados	26
3.10. Síntesis y evidencias de la práctica	27

1. Ítem 1: ICMP y primer contacto con Wireshark

1.1. Fundamentos de ICMP

a) Definición y finalidad. El protocolo ICMP (*Internet Control Message Protocol*) comunica incidencias y permite efectuar diagnósticos de conectividad IPv4. Entre sus mensajes se encuentran destino inalcanzable, tiempo excedido y solicitud/respuesta de eco. No proporciona un servicio de transporte entre aplicaciones mediante puertos, como TCP o UDP; los datos de un eco cumplen una función de diagnóstico. Tampoco garantiza la entrega de los datagramas ni corrige las pérdidas de IP.

b) Relación con IP. ICMP forma parte de las funciones de la capa de red y se encapsula como carga útil de un datagrama IPv4. El receptor reconoce su presencia por el valor 1 del campo *Protocol* del encabezado IPv4. En una trama Ethernet II, el campo *Type* con valor 0x0800 identifica previamente a IPv4.

c) Funcionamiento de ping. La utilidad envía mensajes *Echo Request* y espera sus correspondientes *Echo Reply*; a partir del intercambio estima el tiempo de ida y vuelta (RTT) y la proporción de solicitudes sin respuesta. El campo *Type* vale 8 en la solicitud y 0 en la respuesta; el campo *Code* vale 0 en ambos casos. Recibir una respuesta demuestra que el intercambio ICMP fue posible en ese instante, pero no verifica la disponibilidad de un servicio TCP o UDP.

d) Información mínima del eco. El encabezado ocupa 8 bytes: *Type* (1), *Code* (1), *Checksum* (2), *Identifier* (2) y *Sequence Number* (2). A continuación pueden incluirse datos de longitud variable. El identificador y la secuencia permiten asociar cada respuesta con su solicitud. La especificación exige devolver los mismos datos recibidos en el eco. Estos fundamentos corresponden al RFC 792 (*Internet Control Message Protocol*) y al RFC 791 (*Internet Protocol*).

1.2. Configuración y entorno de medición

La consulta de la configuración con `ip addr` e `ip route` identificó la interfaz inalámbrica que proporciona la ruta a Internet. La tabla registra los valores observados en el equipo de trabajo; las interfaces virtuales de contenedores no se utilizaron para este ensayo.

Parámetro	Valor observado
Interfaz Wi-Fi	wlp5s0
Dirección IPv4	192.168.1.163
Prefijo / máscara	/24 / 255.255.255.0
Red local	192.168.1.0/24
Gateway por defecto	192.168.1.1
MAC de la interfaz	8c:c8:4b:06:70:0b
MAC del gateway (tabla de vecinos)	74:24:9f:b1:e1:cd
Analizador	Wireshark 4.2.2

Cuadro 1: Configuración de la interfaz empleada en el ensayo.

1.3. Procedimiento y resultados generales

El ensayo se realizó el 30 de septiembre de 2026, aproximadamente a las 13:22 h (UTC−3), con Linux y Wireshark 4.2.2. Se inició la captura en wlp5s0, sin modo promiscuo, y se ejecutaron consecutivamente:

```
ping -4 -c 4 -W 2 192.168.1.1
ping -4 -c 4 -W 2 8.8.8.8
```

La opción `-4` fuerza IPv4, `-c 4` limita cada ensayo a cuatro solicitudes y `-W 2` establece la espera de respuesta. Se emplearon direcciones numéricas para evitar consultas DNS. La captura se detuvo automáticamente a los 18 segundos y se conservó en `capturas/icmp_gateway_internet.pcapng`.

Para limitar la adquisición al tráfico propio del experimento se utilizó el siguiente *filtro de captura*, con sintaxis de libpcap:

```
icmp and host 192.168.1.163 and
(host 192.168.1.1 or host 8.8.8.8)
```

Este filtro se introdujo como una sola expresión antes de iniciar la captura. Los filtros de visualización de la consigna se aplicaron posteriormente en Wireshark; no modifican los paquetes almacenados. Sus resultados fueron:

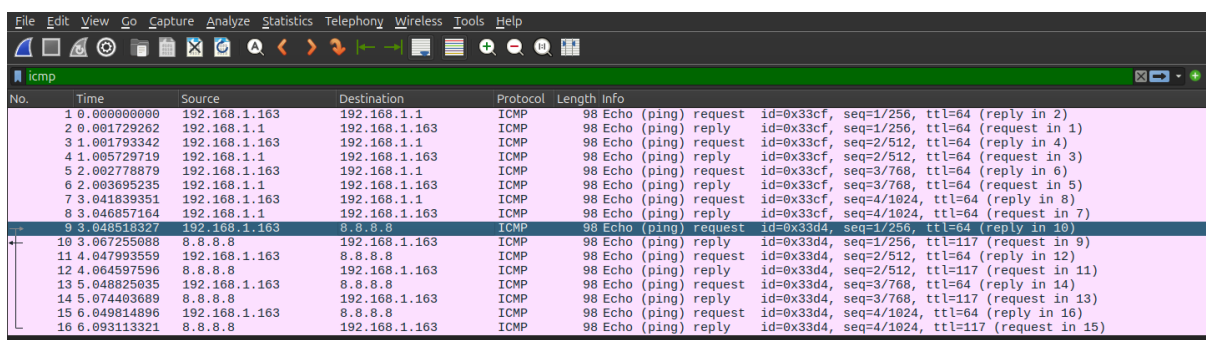
Filtro de visualización	Contenido	Paquetes
icmp	Todos los mensajes ICMP capturados	16
icmp.type == 8	Echo Request	8
icmp.type == 0	Echo Reply	8
icmp && ip.addr == 8.8.8.8	Intercambio con Internet	8

Cuadro 2: Filtros probados sobre el mismo archivo de captura.

Destino	Enviados/recibidos	Pérdida	RTT medio	RTT mín.–máx.
192.168.1.1	4/4	0 %	2,917 ms	0,934–5,035 ms
8.8.8.8	4/4	0 %	26,075 ms	16,627–43,315 ms

Cuadro 3: Resultados informados por ping; se conservaron las salidas originales.

Se registraron ocho pares solicitud–respuesta: las tramas 1–8 corresponden al gateway y las 9–16 a Internet. Wireshark informó 16 paquetes y ninguno descartado durante la captura. Todas las tramas tienen 98 bytes capturados. El mayor RTT hacia Internet es compatible con un recorrido más extenso y con demoras de transmisión, procesamiento y espera; cuatro muestras no permiten caracterizar estadísticamente el desempeño de la red.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.1.163	192.168.1.1	ICMP	98	Echo (ping) request id=0x33cf, seq=1/256, ttl=64 (reply in 2)
2	0.001729262	192.168.1.1	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33cf, seq=1/256, ttl=64 (request in 1)
3	1.001793342	192.168.1.163	192.168.1.1	ICMP	98	Echo (ping) request id=0x33cf, seq=2/512, ttl=64 (reply in 4)
4	1.005729719	192.168.1.1	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33cf, seq=2/512, ttl=64 (request in 3)
5	2.002778879	192.168.1.163	192.168.1.1	ICMP	98	Echo (ping) request id=0x33cf, seq=3/768, ttl=64 (reply in 6)
6	2.003695235	192.168.1.1	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33cf, seq=3/768, ttl=64 (request in 5)
7	3.041839351	192.168.1.163	192.168.1.1	ICMP	98	Echo (ping) request id=0x33cf, seq=4/1024, ttl=64 (reply in 8)
8	3.046857164	192.168.1.1	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33cf, seq=4/1024, ttl=64 (request in 7)
9	3.048518327	192.168.1.163	8.8.8.8	ICMP	98	Echo (ping) request id=0x33d4, seq=1/256, ttl=64 (reply in 10)
10	3.067255088	8.8.8.8	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33d4, seq=1/256, ttl=117 (request in 9)
11	4.047993559	192.168.1.163	8.8.8.8	ICMP	98	Echo (ping) request id=0x33d4, seq=2/512, ttl=64 (reply in 12)
12	4.064597596	8.8.8.8	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33d4, seq=2/512, ttl=117 (request in 11)
13	5.048825035	192.168.1.163	8.8.8.8	ICMP	98	Echo (ping) request id=0x33d4, seq=3/768, ttl=64 (reply in 14)
14	5.074403689	8.8.8.8	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33d4, seq=3/768, ttl=117 (request in 13)
15	6.049814896	192.168.1.163	8.8.8.8	ICMP	98	Echo (ping) request id=0x33d4, seq=4/1024, ttl=64 (reply in 16)
16	6.093113321	8.8.8.8	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33d4, seq=4/1024, ttl=117 (request in 15)

Figura 1: Vista general con el filtro icmp: cuatro pares con el gateway y cuatro con 8.8.8.8. Los números de trama se mantienen en todo el análisis.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.1.163	192.168.1.1	ICMP	98	Echo (ping) request id=0x33cf, seq=1/256, ttl=64 (reply in 2)
2	0.001729262	192.168.1.1	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33cf, seq=1/256, ttl=64 (request in 1)
3	1.001793342	192.168.1.163	192.168.1.1	ICMP	98	Echo (ping) request id=0x33cf, seq=2/512, ttl=64 (reply in 4)
4	1.005729719	192.168.1.1	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33cf, seq=2/512, ttl=64 (request in 3)
5	2.002778879	192.168.1.163	192.168.1.1	ICMP	98	Echo (ping) request id=0x33cf, seq=3/768, ttl=64 (reply in 6)
6	2.003695235	192.168.1.1	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33cf, seq=3/768, ttl=64 (request in 5)
7	3.041839351	192.168.1.163	192.168.1.1	ICMP	98	Echo (ping) request id=0x33cf, seq=4/1024, ttl=64 (reply in 8)
8	3.046857164	192.168.1.1	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33cf, seq=4/1024, ttl=64 (request in 7)
9	3.048518327	192.168.1.163	8.8.8.8	ICMP	98	Echo (ping) request id=0x33d4, seq=1/256, ttl=64 (reply in 10)
10	3.067255088	8.8.8.8	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33d4, seq=1/256, ttl=117 (request in 9)
11	4.047993559	192.168.1.163	8.8.8.8	ICMP	98	Echo (ping) request id=0x33d4, seq=2/512, ttl=64 (reply in 12)
12	4.064597596	8.8.8.8	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33d4, seq=2/512, ttl=117 (request in 11)
13	5.048825035	192.168.1.163	8.8.8.8	ICMP	98	Echo (ping) request id=0x33d4, seq=3/768, ttl=64 (reply in 14)
14	5.074403689	8.8.8.8	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33d4, seq=3/768, ttl=117 (request in 13)
15	6.049814896	192.168.1.163	8.8.8.8	ICMP	98	Echo (ping) request id=0x33d4, seq=4/1024, ttl=64 (reply in 16)
16	6.093113321	8.8.8.8	192.168.1.163	ICMP	98	Echo (ping) reply id=0x33d4, seq=4/1024, ttl=117 (request in 15)

Figura 2: Aplicación de los filtros de solicitudes, respuestas e intercambio con Internet, respectivamente. Cada vista contiene ocho paquetes; los saltos en la numeración se deben al filtrado.

1.4. Identificación de las capas y sus campos

Se eligió la trama 9, primera solicitud a 8.8.8.8. La tabla distingue las direcciones de cada capa de los campos que identifican el eco. ICMP no posee direcciones ni puertos propios: el identificador y la secuencia corresponden al intercambio, no a un origen y un destino independientes.

Capa en Wireshark	Origen / identificador	Destino / identificador	Campo que identifica el contenido
Ethernet II	8c:c8:4b:06:70:0b	74:24:9f:b1:e1:cd	<i>Type</i> : 0x0800, IPv4.
Internet Protocol Version 4	192.168.1.163	8.8.8.8	<i>Protocol</i> : 1, ICMP.
Internet Control Message Protocol	Sin dirección propia; identificador 0x33d4	Sin dirección propia; secuencia 1	<i>Type</i> =8 y <i>Code</i> =0 identifican Echo Request; no existe un campo que indique otro protocolo de transporte interno.
Datos / payload	Sin dirección propia	Sin dirección propia	56 bytes de datos de diagnóstico; no identifican un protocolo encapsulado.

Cuadro 4: Encapsulación y direccionamiento de la trama 9.

Aunque la interfaz física es Wi-Fi, el controlador entrega a la captura una representación Ethernet II en modo de operación normal. Por ello no se observan aquí encabezados 802.11 ni Radiotap. Además, *Frame* es la información de captura que agrega Wireshark, no un encabezado transmitido.

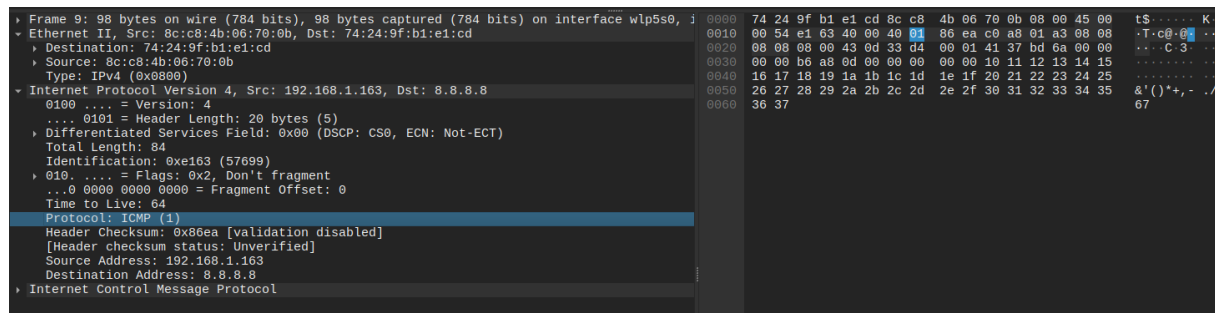


Figura 3: Detalles de la trama 9: *Type*=0x0800 en Ethernet II y *Protocol*=1 en IPv4. La selección de *Protocol* resalta el byte 01 en el panel hexadecimal.

1.5. a) MAC destino y alcance del direccionamiento

En la trama 9, la IP destino es 8.8.8.8, pero la MAC destino es 74:24:9f:b1:e1:cd. Esta última pertenece a la interfaz local del gateway 192.168.1.1, según la tabla de vecinos. La trama 1, dirigida a la propia IP del gateway, contiene exactamente la misma MAC destino. La consulta `ip route get 8.8.8.8` confirmó que el siguiente salto es 192.168.1.1 a través de wlp5s0.

Con la máscara /24, 8.8.8.8 está fuera de la subred local. Por ello, el equipo entrega

la trama al router para que reenvíe el datagrama. La MAC identifica al receptor de la entrega en el enlace local; la IP identifica al destino del datagrama a través de redes interconectadas. Cada salto ruteado utiliza un nuevo encabezado de enlace, de acuerdo con el enlace de salida. Esta captura no permite conocer la MAC del equipo remoto que responde por 8.8.8.8.

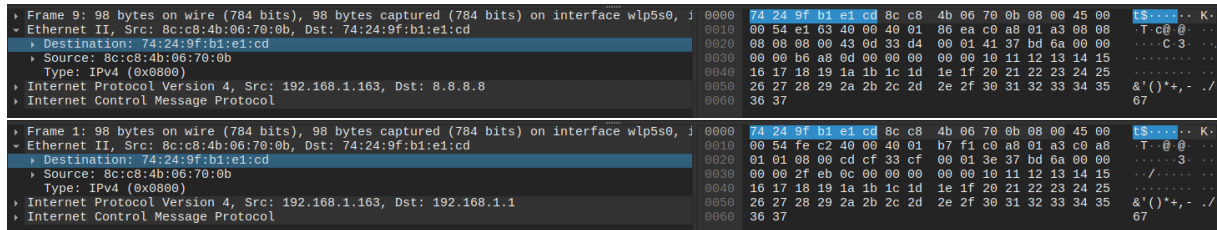


Figura 4: Comparación de la MAC destino resaltada: solicitud a Internet (trama 9, arriba) y al gateway (trama 1, abajo). Los seis primeros bytes son 74 24 9f b1 e1 cd en ambos casos.

1.6. b) Comparación entre Echo Request y Echo Reply

Wireshark vincula la trama 9 con la 10 mediante *[Response frame: 10]*; en la respuesta aparece *[Request frame: 9]*. La diferencia entre sus marcas temporales es 18,736761 ms, presentada por Wireshark como 18,737 ms. Este intervalo entre paquetes capturados puede diferir ligeramente del RTT de ping (18,8 ms para ese eco), que utiliza sus propias marcas temporales y redondeo.

Campo que cambia	Request (trama 9)	Reply (trama 10)
Ethernet: MAC origen	8c:c8:4b:06:70:0b	74:24:9f:b1:e1:cd
Ethernet: MAC destino	74:24:9f:b1:e1:cd	8c:c8:4b:06:70:0b
IPv4: origen	192.168.1.163	8.8.8.8
IPv4: destino	8.8.8.8	192.168.1.163
IPv4: identificación	0xe163 (57699)	0x0000 (0)
IPv4: flags	0x2: DF=1	0x0: DF=0
IPv4: TTL	64	117
IPv4: checksum	0x86ea	0x734e
ICMP: tipo	8 (Echo Request)	0 (Echo Reply)
ICMP: checksum	0x430d	0x4b0d

Cuadro 5: Campos diferentes entre la primera solicitud a Internet y su respuesta.

Capa	Campos y longitudes que se mantienen
Ethernet II	Encabezado de 14 bytes, <i>Type</i> =0x0800 y longitud capturada de la trama de 98 bytes.
IPv4	Versión 4; IHL=5 (20 bytes, sin opciones); longitud total 84 bytes; DSCP=0 y ECN=0; protocolo 1; bit reservado=0, MF=0 y desplazamiento de fragmento=0. No se observa fragmentación.
ICMP	Código 0; identificador 0x33d4 (13268); secuencia 1; encabezado de 8 bytes y mensaje de 64 bytes. Ambos checksums ICMP se validan como correctos.
Payload	Los 56 bytes se conservan íntegramente, incluida la marca temporal.

Cuadro 6: Características comunes de las tramas 9 y 10.

Las direcciones se invierten por el cambio de sentido de la comunicación; en Ethernet la respuesta llega desde el gateway, que es el vecino del equipo en este enlace. El emisor remoto construye un nuevo datagrama IPv4: no tiene que copiar la identificación ni la política DF de la solicitud. Sus valores observados describen este intercambio y no una regla universal para todas las implementaciones. El TTL depende del valor inicial del emisor y del recorrido, como se analiza en el apartado d).

El checksum IPv4 cambia porque protege un encabezado cuyos campos han cambiado. En ICMP, la sustitución del tipo 8 por 0 exige recalcular el checksum aun cuando el resto del mensaje de eco se conserve. La interfaz de Wireshark mostró la validación del checksum IPv4 deshabilitada; ello no indica por sí mismo un error del paquete. En cambio, los dos checksums ICMP aparecen como *correct*.

El identificador y la secuencia se conservan para reconocer a qué solicitud corresponde cada respuesta, incluso con varios ecos pendientes. No deben confundirse con la identificación de IPv4. Wireshark ofrece interpretaciones *BE* y *LE* de los mismos dos bytes; en este informe se utiliza *BE*, por lo que 00 01 representa la secuencia 1, no 256. Los vínculos entre tramas y el tiempo de respuesta entre corchetes son metadatos calculados por Wireshark: no ocupan bytes en el paquete.

```

Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Checksum: 0x430d [correct]
  [Checksum Status: Good]
  Identifier (BE): 13268 (0x33d4)
  Identifier (LE): 54323 (0xd433)
  Sequence Number (BE): 1 (0x0001)
  Sequence Number (LE): 256 (0x0100)
  [Response frame: 10]
  Timestamp from icmp data: Sep 30, 2026 13:22:25.895158000 -03
  [Timestamp from icmp data (relative): 0.000017915 seconds]
  Data (40 bytes)

```

Figura 5: Encabezado ICMP de la trama 9 y vínculo a la respuesta 10. El identificador y la secuencia se leen como valores *big-endian*.

1.7. c) Ubicación, tamaño y contenido del payload

El payload de `ping` comienza inmediatamente después del encabezado ICMP, en el desplazamiento decimal 42 (0x002a) desde el inicio de la trama: $14 + 20 + 8 = 42$. Ocupa 56 bytes, hasta el desplazamiento 97 inclusive. Es importante distinguir esta cantidad de los 64 bytes del mensaje ICMP completo, de los 84 del datagrama IPv4 y de los 98 de la trama capturada.

En esta ejecución de `ping` de `iputils` sobre Linux de 64 bits, Wireshark interpreta los primeros 16 bytes del payload como una marca temporal. En la trama 9 corresponde a las 13:22:25,895158 h (UTC-3). Los 40 bytes restantes forman el patrón incremental desde 0x10 hasta 0x37. Por eso Wireshark muestra *Data (40 bytes)*: los otros 16 bytes también pertenecen al payload, pero se decodifican por separado como *Timestamp from icmp data*. El manual local `man 8 ping`, en *ICMP PACKET DETAILS*, describe el uso de una estructura `timeval` al comienzo de los datos.

```
41 37 bd 6a 00 00 00 00 b6 a8 0d 00 00 00 00 00
10 11 12 13 14 15 16 17 18 19 1a 1b 1c 1d 1e 1f
20 21 22 23 24 25 26 27 28 29 2a 2b 2c 2d 2e 2f
30 31 32 33 34 35 36 37
```

Figura 6: Transcripción hexadecimal de los 56 bytes del payload de la trama 9, idénticos a los de la trama 10. La primera línea contiene la marca temporal.

La comparación byte a byte confirmó la igualdad del payload entre solicitud y respuesta en los ocho pares capturados. La marca temporal se devuelve sin reemplazarla por la hora del receptor, lo que permite al emisor estimar el RTT. No se dispuso de una captura de Windows en este ensayo, por lo que no se presenta una comparación experimental entre sistemas operativos. La longitud y el patrón de relleno dependen de la implementación y las opciones de `ping`; ICMP no exige este patrón concreto. Por tanto, observar datos diferentes entre implementaciones sería compatible con el protocolo, siempre que cada respuesta devolviera los datos de su solicitud.

```
▼ Internet Control Message Protocol
  Type: 8 (Echo (ping) request)
  Code: 0
  Checksum: 0x430d [correct]
  [Checksum Status: Good]
  Identifier (BE): 13268 (0x33d4)
  Identifier (LE): 54323 (0xd433)
  Sequence Number (BE): 1 (0x0001)
  Sequence Number (LE): 256 (0x0100)
  [Response frame: 10]
  Timestamp from icmp data: Sep 30, 2026 13:22:25.895158000 -03
  [Timestamp from icmp data (relative): 0.000017915 seconds]
  ▼ Data (40 bytes)
    Data: 101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f30313233343536
    [Length: 40]

▼ Internet Control Message Protocol
  Type: 0 (Echo (ping) reply)
  Code: 0
  Checksum: 0x4b0d [correct]
  [Checksum Status: Good]
  Identifier (BE): 13268 (0x33d4)
  Identifier (LE): 54323 (0xd433)
  Sequence Number (BE): 1 (0x0001)
  Sequence Number (LE): 256 (0x0100)
  [Request frame: 9]
  [Response time: 18.737 ms]
  Timestamp from icmp data: Sep 30, 2026 13:22:25.895158000 -03
  [Timestamp from icmp data (relative): 0.018754676 seconds]
  ▼ Data (40 bytes)
    Data: 101112131415161718191a1b1c1d1e1f202122232425262728292a2b2c2d2e2f30313233343536
    [Length: 40]
```

Figura 7: Detalle ICMP de la solicitud 9 (arriba) y la respuesta 10 (abajo): marca temporal y bloque *Data* de 40 bytes. El payload completo tiene 56 bytes, como confirma la longitud del datagrama.

1.8. d) Análisis del TTL

La solicitud de la trama 9 sale del equipo con TTL=64, mientras que la respuesta de la trama 10 llega desde 8.8.8.8 con TTL=117. Estos valores se repiten en los cuatro pares con ese destino. En el intercambio con el gateway se observan TTL=64 tanto en la solicitud como en la respuesta.

Al reenviar un datagrama IPv4, un router reduce su TTL al menos en una unidad; si este se agota, descarta el datagrama, evitando su circulación indefinida. Este comportamiento se especifica en el RFC 791 y se desarrolla para routers en el RFC 1812, sección 5.3.1. El Echo Reply es un datagrama nuevo con un TTL inicial elegido por el emisor remoto; no hereda el TTL de la solicitud. Por ello, un Reply puede llegar con un TTL superior al del Request observado al salir.

La captura local no permite conocer el TTL inicial del Reply ni el número exacto de saltos recorridos. Si se supusiera un valor inicial de 128 y una reducción de una unidad por salto, 117 sería compatible con 11 saltos de retorno; esto es una hipótesis, no una medición del trayecto. Tampoco puede suponerse que las rutas de ida y vuelta sean iguales.

```

▶ Frame 9: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface wlp5s0, 1
▼ Ethernet II, Src: 8c:c8:4b:06:70:0b, Dst: 74:24:9f:b1:e1:cd
  ▶ Destination: 74:24:9f:b1:e1:cd
  ▶ Source: 8c:c8:4b:06:70:0b
  Type: IPv4 (0x0800)
▼ Internet Protocol Version 4, Src: 192.168.1.163, Dst: 8.8.8.8
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 84
  Identification: 0xe163 (57699)
  ▶ 010. .... = Flags: 0x2, Don't fragment
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 64
  Protocol: ICMP (1)
  Header Checksum: 0x86ea [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 192.168.1.163
  Destination Address: 8.8.8.8
  Internet Control Message Protocol
.....
▶ Frame 10: 98 bytes on wire (784 bits), 98 bytes captured (784 bits) on interface wlp5s0,
▼ Ethernet II, Src: 74:24:9f:b1:e1:cd, Dst: 8c:c8:4b:06:70:0b
  ▶ Destination: 8c:c8:4b:06:70:0b
  ▶ Source: 74:24:9f:b1:e1:cd
  Type: IPv4 (0x0800)
▼ Internet Protocol Version 4, Src: 8.8.8.8, Dst: 192.168.1.163
  0100 .... = Version: 4
  .... 0101 = Header Length: 20 bytes (5)
  ▶ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
  Total Length: 84
  Identification: 0x0000 (0)
  ▶ 000. .... = Flags: 0x0
  ...0 0000 0000 0000 = Fragment Offset: 0
  Time to Live: 117
  Protocol: ICMP (1)
  Header Checksum: 0x734e [validation disabled]
  [Header checksum status: Unverified]
  Source Address: 8.8.8.8
  Destination Address: 192.168.1.163
  Internet Control Message Protocol

```

Figura 8: TTL resaltado en IPv4: 64 en la solicitud 9 (arriba) y 117 en la respuesta 10 (abajo). La identificación de la trama y las direcciones permiten distinguir ambos sentidos.

1.9. e) Encapsulación y distribución de bytes

La figura representa los tamaños observados en la trama 9, sin opciones IPv4 ni etiquetas VLAN. Los tamaños de las cajas interiores están incluidos en los de las exteriores: no deben sumarse como longitudes independientes.

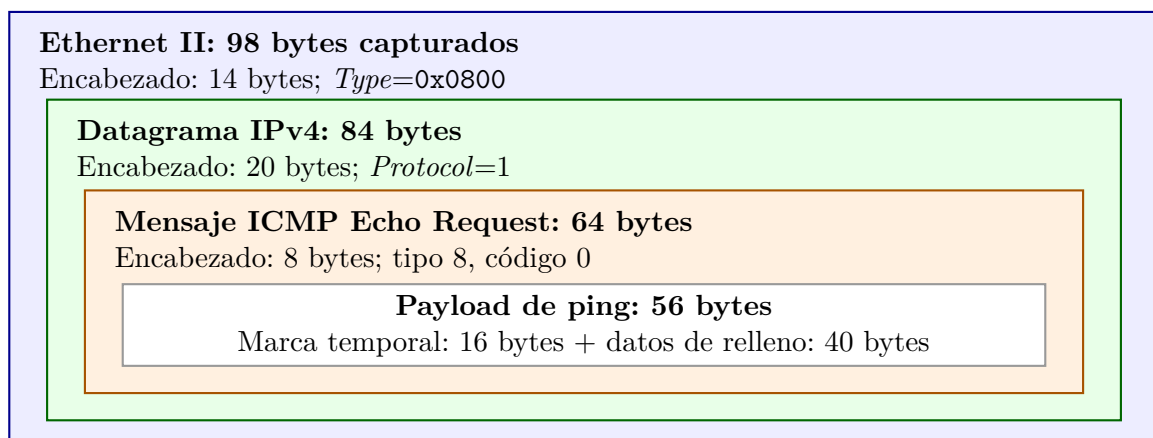


Figura 9: Encapsulación de la trama 9 mediante cajas anidadas.

Componente	Inicio	Fin	Longitud
Encabezado Ethernet II	0	13	14 bytes
Encabezado IPv4	14	33	20 bytes
Encabezado ICMP Echo	34	41	8 bytes
Marca temporal en el payload	42	57	16 bytes
Resto del payload	58	97	40 bytes

Cuadro 7: Desplazamientos decimales, inclusivos, desde el inicio de la trama capturada.

La suma es $14 + 20 + 8 + 56 = 98$ bytes. La captura no incluye FCS Ethernet, preámbulo ni intervalo entre tramas, ni expone la sobrecarga física de 802.11; por tanto, 98 bytes no representa el costo total de transmisión por el medio inalámbrico. Se trata de la longitud de la representación de enlace entregada al analizador.

1.10. Síntesis y evidencias de la práctica

El ensayo permitió verificar la encapsulación Ethernet II–IPv4–ICMP, la diferencia entre el siguiente salto de enlace y el destino IP, y la correspondencia entre solicitudes y respuestas mediante identificador, secuencia y datos. El destino de Internet respondió a las cuatro solicitudes, por lo que no fue necesario limitar el análisis al gateway.

Las evidencias se conservan en las carpetas del informe:

- `capturas/icmp_gateway_internet.pcapng`: captura original, con los 16 paquetes y su numeración.
- `capturas/`: salidas de `ping`, configuración de red y `analisis_wireshark.json`, exportado mediante `sharkd` para verificar campos y payloads.
- `img/icmp_*.png`: doce capturas originales de Wireshark. El informe recorta paneles para facilitar su lectura, sin alterar los campos.

Fuentes técnicas: RFC 792, 791 y 1812; manual local `ping(8)` de `iputils`; manual de Wireshark, capítulo 6. Los resultados del ensayo proceden de la captura adjunta.

2. Ítem 2: ARP: de una IP a una dirección MAC

2.1. Fundamentos de ARP y resolución de direcciones

a) **Problema que resuelve ARP y ubicación en el modelo en capas.** El protocolo ARP (*Address Resolution Protocol*, formalizado en el RFC 826) resuelve el problema del mapeo dinámico entre direcciones lógicas de capa de red (direcciones IPv4 de 32 bits) y direcciones físicas de capa de enlace (direcciones MAC IEEE 802 de 48 bits) dentro de un mismo segmento de red local o dominio de difusión. Cuando un equipo emisor necesita transmitir un datagrama IPv4 hacia otro nodo situado en su misma subred (o hacia su gateway por defecto si el destino es externo), la capa de aplicación y el subsistema IP solo conocen la dirección IP del destinatario. Sin embargo, las interfaces de red Ethernet y Wi-Fi en el medio físico únicamente pueden conmutar y entregar tramas utilizando direcciones MAC de capa de enlace. ARP proporciona el mecanismo estandarizado para descubrir qué dirección física corresponde a una dirección IP determinada.

Determinar en qué capa se ubica ARP es un tema clásicamente discutible:

- Por un lado, no pertenece estrictamente a la capa de enlace (Capa 2 del modelo OSI) porque maneja conceptos propios de la capa de red (conoce y transporta direcciones IPv4 de 32 bits y tipos de protocolo de capa superior).
- Por otro lado, tampoco encaja estrictamente en la capa de red (Capa 3) debido a que no proporciona funciones de enrutamiento ni viaja encapsulado dentro de un datagrama IP; sus paquetes se colocan directamente en la carga útil de la trama Ethernet (con valor de *EtherType* 0x0806).

Por estas razones, se lo clasifica habitualmente como un protocolo de “capa 2.5” o una función auxiliar de enlace y red que opera como intermediario o interfaz entre ambas capas.

b) Mensajes ARP Request y ARP Reply.

- *ARP Request* (Solicitud ARP): Es un mensaje de consulta emitido por el host que requiere conocer la MAC asociada a una IP específica (“¿Quién tiene la dirección IP *X*? Respóndale a *Y*”). Debido a que el emisor desconoce qué máquina física de la red posee dicha dirección IP, la trama Ethernet se envía a la dirección física de difusión (*broadcast*, ff:ff:ff:ff:ff:ff). De este modo, todas las estaciones del segmento local reciben y procesan la solicitud. En el encabezado ARP, el campo *Opcode* vale 1 (Request) y el campo *Target MAC address* se inicializa con ceros (00:00:00:00:00:00), ya que constituye precisamente la incógnita a resolver.
- *ARP Reply* (Respuesta ARP): Es el mensaje generado exclusivamente por el nodo que posee la dirección IP solicitada (“Yo tengo la dirección IP *X*, y mi dirección MAC es *Z*”). A diferencia de la solicitud, la respuesta se envía de forma directa y unívoca (*unicast*) hacia la dirección MAC del equipo que originó la consulta, la cual fue aprendida a partir de los campos *Sender MAC* y *Sender IP* de la solicitud recibida. En el encabezado ARP, el campo *Opcode* vale 2 (Reply), *Sender MAC*

contiene la dirección física del nodo que responde y *Target MAC* contiene la dirección física del solicitante.

c) La caché ARP y su función. La caché ARP es una tabla en memoria administrada por el núcleo del sistema operativo donde se almacenan las asociaciones dinámicas (y eventualmente estáticas) aprendidas recientemente entre direcciones IPv4 y sus correspondientes direcciones MAC. Su existencia responde a dos necesidades fundamentales de ingeniería de redes:

1. *Rendimiento y reducción de latencia:* Evita que el emisor deba ejecutar un ciclo completo de consulta y espera ARP cada vez que envía un paquete IP a un mismo destino. Al disponer de la entrada en la memoria local, el encapsulado de la trama Ethernet se produce de forma inmediata.
2. *Eficiencia y reducción del tráfico de difusión:* Cada solicitud ARP por broadcast obliga a todos los hosts del enlace a interrumpir sus procesadores para analizar el contenido del paquete. Almacenar temporalmente las resoluciones reduce drásticamente el consumo de ancho de banda y la saturación del medio físico.

Las entradas dinámicas poseen un tiempo de vida finito (*aging time* o TTL de caché, típicamente de unos pocos minutos según el sistema operativo), garantizando que las asociaciones obsoletas expiren para adaptarse a cambios en la topología (tales como reemplazos de adaptadores de red, movilidad de dispositivos o reasignación de direcciones por DHCP).

d) Proceso de resolución expresado en términos prácticos. Si dispongo de la dirección IP de una máquina perteneciente a mi red local, el procedimiento que sigue mi sistema operativo para determinar la dirección MAC a la cual enviar la trama Ethernet es el siguiente:

1. Primero consulta su tabla o caché ARP local para verificar si ya existe una correspondencia válida entre esa dirección IP y una dirección física.
2. Si la entrada existe (*cache hit*), extrae la dirección MAC asociada de forma instantánea y construye la trama Ethernet colocando dicha dirección en el campo de destino.
3. Si la entrada no existe (*cache miss*), el sistema operativo retiene temporalmente el datagrama IP en una cola de espera y emite un mensaje *ARP Request* enviado en difusión (*broadcast*, `ff:ff:ff:ff:ff:ff`) a toda la red local preguntando quién posee esa IP.
4. Todos los nodos de la red reciben la trama, pero únicamente el equipo con la IP buscada contesta mediante un mensaje *ARP Reply* enviado por *unicast* directamente a mi máquina, adjuntando su dirección MAC física.
5. Al recibir esta respuesta, mi sistema operativo guarda la nueva asociación en su caché ARP para usos posteriores, desencola el datagrama IP pendiente y lo encapsula en la trama Ethernet dirigida a la MAC recién descubierta.

2.2. Inspección de la caché ARP local

Para verificar el contenido de la caché de resolución de direcciones en el entorno de trabajo, se ejecutó el comando `arp -a` en una terminal del sistema Windows. La figura 10 reproduce la salida obtenida.

```
C:\Users\rocco>arp -a

Interfaz: 192.168.0.219 --- 0x8
Dirección de Internet      Dirección física      Tipo
192.168.0.1                34-53-d2-98-26-47    dinámico
192.168.0.255              ff-ff-ff-ff-ff-ff    estático
224.0.0.22                 01-00-5e-00-00-16    estático
224.0.0.251                01-00-5e-00-00-fb    estático
224.0.0.252                01-00-5e-00-00-fc    estático
239.255.255.250            01-00-5e-7f-ff-fa    estático
255.255.255.255            ff-ff-ff-ff-ff-ff    estático

Interfaz: 192.168.56.1 --- 0xb
Dirección de Internet      Dirección física      Tipo
192.168.56.255             ff-ff-ff-ff-ff-ff    estático
224.0.0.22                 01-00-5e-00-00-16    estático
224.0.0.251                01-00-5e-00-00-fb    estático
224.0.0.252                01-00-5e-00-00-fc    estático
239.255.255.250            01-00-5e-7f-ff-fa    estático
```

Figura 10: Visualización de la tabla de vecinos y caché ARP mediante `arp -a` en el sistema operativo Windows.

Interfaz local	Dirección de Internet (IPv4)	Dirección física (MAC)	Tipo
192.168.0.219	192.168.0.1 (Gateway)	34-53-d2-98-26-47	Dinámico
192.168.0.219	192.168.0.255 (Broadcast subred)	ff-ff-ff-ff-ff-ff	Estático
192.168.0.219	224.0.0.22 (Multicast IGMP)	01-00-5e-00-00-16	Estático
192.168.0.219	224.0.0.251 (mDNS)	01-00-5e-00-00-fb	Estático
192.168.0.219	224.0.0.252 (LLMNR)	01-00-5e-00-00-fc	Estático
192.168.0.219	239.255.255.250 (SSDP)	01-00-5e-7f-ff-fa	Estático
192.168.0.219	255.255.255.255 (Broadcast general)	ff-ff-ff-ff-ff-ff	Estático
192.168.56.1	192.168.56.255 (Broadcast host)	ff-ff-ff-ff-ff-ff	Estático

Cuadro 8: Entradas registradas en la caché ARP de las interfaces de red del equipo de ensayo.

Coincidencia de la MAC del gateway. Al consultar la interfaz activa principal (192.168.0.219), se identifica la entrada correspondiente a la dirección IPv4 del router/gateway por defecto (192.168.0.1), asociada a la dirección física 34-53-d2-98-26-47 como entrada de tipo *dinámico*.

Esta dirección física coincide exactamente con la dirección MAC de destino utilizada por la tarjeta de red en el encabezado de enlace Ethernet cada vez que el equipo transmite un paquete hacia el gateway o hacia cualquier destino externo fuera de la subred local (como redes de Internet). En el ensayo del Ítem 1 llevado a cabo sobre una estación Linux

del grupo de trabajo, la dirección IP del gateway fue 192.168.1.1 y su dirección MAC registrada fue 74:24:9f:b1:e1:cd. En ambos casos se confirma la misma regla operativa fundamental de las redes TCP/IP: la dirección MAC de destino de la trama Ethernet de enlace local es invariablemente la MAC del gateway por defecto obtenida mediante resolución ARP.

Adicionalmente, la tabla evidencia el mapeo de direcciones IP de difusión (*broadcast*) a la dirección física ff-ff-ff-ff-ff-ff, y de direcciones IP de multidifusión (*multicast*, rango 224.0.0.0/4) al prefijo Ethernet asignado por IANA 01-00-5e-xx-xx-xx. La segunda interfaz observada (192.168.56.1) corresponde a un adaptador virtual de máquina local (VirtualBox).

2.3. Generación de tráfico ARP y captura con Wireshark

Debido a que el sistema operativo almacena las asociaciones en su caché durante varios minutos, un comando `ping` dirigido a un nodo con el que ya se mantuvo comunicación previa no generará tráfico ARP visible en la red. Para provocar y estudiar el intercambio ARP de forma controlada, se contemplan los métodos descritos en la guía de trabajos prácticos:

- **Opción A (Sondeo a IP inexistente en la subred local):** Se envía un `ping` a una dirección IP perteneciente al mismo bloque de red pero que no esté asignada a ningún equipo en funcionamiento (por ejemplo, `ping -4 -n 2 192.168.0.253` en una subred 192.168.0.0/24). Como la IP no figura en la caché, el equipo genera múltiples mensajes *ARP Request* en difusión para intentar localizarla. Al no haber ningún host que responda, no se produce ningún *ARP Reply* y el comando finaliza indicando “Host de destino inaccesible”. Este método tiene la ventaja de no requerir privilegios de administrador.
- **Opción B (Invalidación temporal de la entrada del gateway):** Se elimina la asociación correspondiente al router mediante permisos de administrador (`arp -d 192.168.0.1` en Windows o `sudo ip neigh del 192.168.1.1 dev wlan0` en Linux) y de inmediato se efectúa un `ping` a dicha dirección. Esto fuerza al sistema a emitir un *ARP Request* en difusión, al cual el gateway responde inmediatamente con un *ARP Reply* unicast, permitiendo capturar el par completo solicitud-respuesta antes de enviar el tráfico ICMP.
- **Opción C (Comunicación con otro host de la LAN):** Se realiza un `ping` hacia otra estación de trabajo presente en la red local con la cual no se haya interactuado recientemente.

Para aislar y examinar los paquetes capturados en Wireshark se emplean los siguientes filtros de visualización:

- `arp`: Visualiza todos los paquetes del protocolo ARP.
- `arp.opcode == 1`: Filtra exclusivamente los mensajes de solicitud (*ARP Request*).
- `arp.opcode == 2`: Filtra exclusivamente los mensajes de respuesta (*ARP Reply*).

- `arp.dst.proto_ipv4 == 192.168.0.1`: Filtra consultas dirigidas a conocer la MAC del gateway.
- `arp || icmp`: Muestra conjuntamente las consultas ARP y los mensajes ICMP generados por la herramienta de diagnóstico.
- `eth.dst == ff:ff:ff:ff:ff:ff`: Aísla todas las tramas enviadas a nivel de enlace a la dirección de difusión.

2.4. Análisis comparativo de ARP Request y ARP Reply

En la práctica se ejecutó el procedimiento de la Opción A enviando un ping a la dirección no asignada 192.168.0.253. La figura 11 presenta el paquete capturado en Wireshark (trama 5833), correspondiente a la solicitud *ARP Request* generada por el equipo local.

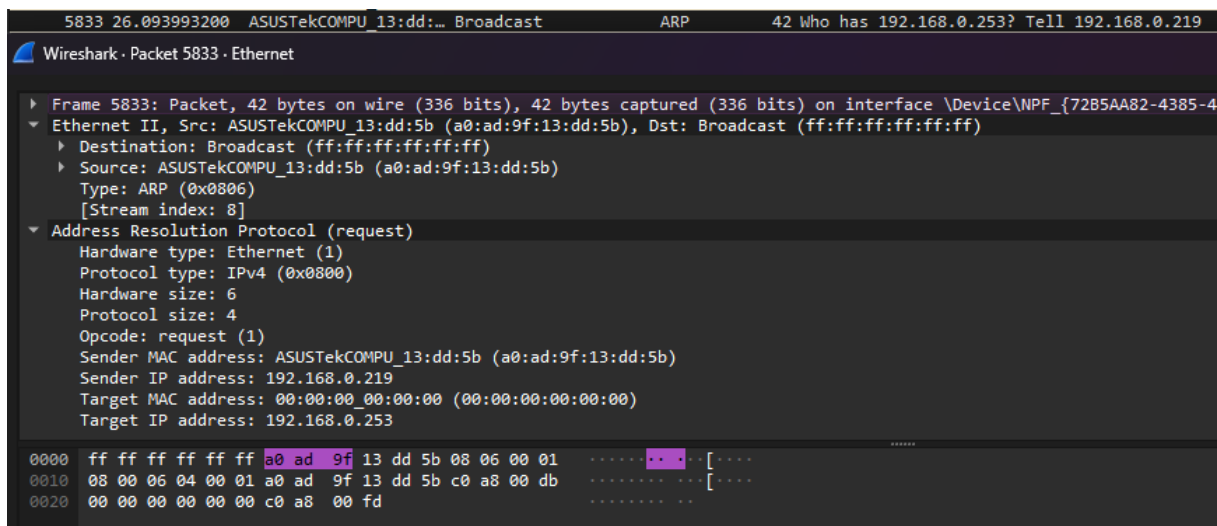


Figura 11: Detalle en Wireshark de la solicitud ARP (trama 5833): consulta en difusión (*broadcast*) enviada por la interfaz local (192.168.0.219) solicitando la dirección física de la IP inexistente 192.168.0.253.

Como se aprecia en la captura, la trama capturada tiene una longitud total de 42 bytes (14 bytes del encabezado Ethernet II y 28 bytes del mensaje ARP). El volcado hexadecimal inferior resalta la estructura completa: los primeros 6 bytes corresponden a la MAC de difusión `ff ff ff ff ff ff`, seguidos por la MAC de origen del adaptador ASUS (`a0 ad 9f 13 dd 5b`) y el identificador de protocolo `08 06`. A continuación, el cuerpo ARP especifica *Hardware type*=1 (Ethernet), *Protocol type*=0x0800 (IPv4), longitudes 6 y 4 bytes, y *Opcode*=00 01 (*request*).

A partir de la trama obtenida, la tabla 9 detalla los campos de nivel de enlace y de protocolo para la solicitud capturada, comparándolos con los valores que adopta una respuesta *ARP Reply* cuando el destino responde de forma efectiva en la red local.

Campo analizado	ARP Request (Trama 5833)	ARP Reply (Caso con respuesta)
MAC destino (encabezado Ethernet)	ff:ff:ff:ff:ff:ff (Broadcast)	a0:ad:9f:13:dd:5b (Unicast al solicitante)
MAC origen (encabezado Ethernet)	a0:ad:9f:13:dd:5b (MAC del emisor)	MAC física del respondedor
EtherType (encabezado Ethernet)	0x0806 (ARP)	0x0806 (ARP)
Hardware Type / Protocol Type	1 (Ethernet) / 0x0800 (IPv4)	1 (Ethernet) / 0x0800 (IPv4)
Hardware Size / Protocol Size	6 bytes / 4 bytes	6 bytes / 4 bytes
Opcode	1 (<i>request</i>)	2 (<i>reply</i>)
Sender MAC address	a0:ad:9f:13:dd:5b	MAC física del respondedor
Sender IP address	192.168.0.219	IP del respondedor (ej. 192.168.0.1)
Target MAC address	00:00:00:00:00:00 (Desconocida)	a0:ad:9f:13:dd:5b (MAC del solicitante)
Target IP address	192.168.0.253	192.168.0.219 (IP del solicitante)

Cuadro 9: Comparación detallada de campos entre la solicitud capturada y la estructura de una respuesta ARP.

Nota sobre el ensayo: Al haberse empleado la Opción A (sondeo a la dirección libre 192.168.0.253), **no se produjo ningún mensaje *ARP Reply*** en la red debido a que ningún nodo físico posee asignada dicha IP. La columna de *ARP Reply* de la tabla 9 presenta la estructura canónica esperada para una resolución exitosa dentro de la misma subred (tal como se verificó previamente con la entrada del gateway en la caché).

a) Justificación de difusión vs. entrega unívoca y valor de *Target MAC address*.

- El mensaje *ARP Request* debe enviarse forzosamente a una dirección de difusión (*broadcast*, ff:ff:ff:ff:ff:ff) debido a que el host emisor desconoce totalmente qué placa física posee la IP consultada. La difusión garantiza que los conmutadores (*switches*) inunden la trama hacia todos los puertos del dominio de colisión y que todas las tarjetas de red la acepten y la eleven a sus pilas de protocolos para su verificación.
- Por el contrario, el mensaje *ARP Reply* se transmite de forma unívoca (*unicast*) directamente a la dirección MAC del solicitante, dado que dicha dirección ya fue aprendida de los campos *Sender MAC* y *Sender IP* incluidos en el Request. No existe necesidad de molestar al resto de los integrantes de la red con una respuesta que concierne a un único receptor.
- En el paquete *ARP Request*, el campo *Target MAC address* posee el valor en ceros 00:00:00:00:00:00 (visible en el desplazamiento 0x0020 del panel hexadecimal

como 00 00 00 00 00 00). Esto se debe a que la dirección física del destino es el dato desconocido que se busca determinar; al no conocerla, la especificación del protocolo estipula completarla con ceros como marcador de posición.

b) Encabezado Ethernet, campo *Type* y ausencia de encabezado IP. En el encabezado de la trama Ethernet que transporta un mensaje ARP, el campo *Type* (*EtherType*) tiene asignado el valor hexadecimal 0x0806, correspondiente a ARP, a diferencia del valor 0x0800 reservado para IPv4. Dentro de la trama capturada **no existe ningún encabezado IP**. El paquete ARP se coloca directamente en el campo de carga útil (*payload*) de la trama Ethernet II. Esto demuestra empíricamente que ARP no es un protocolo de capa de aplicación ni viaja sobre la capa de red: “vive” como un protocolo auxiliar directamente sobre la capa de enlace de datos, cumpliendo una función de traducción que enlaza ambas capas.

c) Comportamiento observado ante una IP inexistente (Opción A). En la práctica realizada con la IP inexistente 192.168.0.253:

1. En la captura de Wireshark se observan solicitudes *ARP Request* sucesivas (como la trama 5833), generadas por el sistema operativo que reintenta resolver la dirección tras vencer sus temporizadores de espera (*timeouts*).
2. **No se produjo ningún *ARP Reply***, puesto que ninguna máquina conectada al medio posee asignada dicha dirección IP.
3. **No apareció ningún paquete *ICMP Echo Request* en la captura.** Esta observación es sumamente reveladora: la capa de red genera el datagrama ICMP internamente, pero al solicitar a la capa de enlace que lo encapsule y lo transmita, esta no puede construir una trama Ethernet válida al carecer de una dirección MAC de destino. El datagrama queda retenido en los búferes del sistema operativo a la espera de la resolución física; al vencer el tiempo sin obtener respuesta ARP, el paquete es finalmente descartado por el kernel y la consola informa “Host de destino inaccesible” (*Destination host unreachable*). El datagrama ICMP jamás llega a transmitirse por el medio físico.

d) Reintento de comunicación, vigencia de la caché y análisis de fallos. Al repetir el comando `ping` a un destino operativo transcurrido un lapso breve (por ejemplo, un minuto después de una resolución exitosa con el gateway), **no se observa tráfico ARP en la captura**. El sistema operativo consulta su caché ARP local, detecta que la entrada sigue vigente y procede de inmediato a transmitir los datagramas ICMP sobre tramas Ethernet sin generar ningún intercambio de resolución previo. En cambio, para una dirección fallida que no respondió, el sistema operativo no retiene una correspondencia MAC válida, por lo que un nuevo intento volverá a gatillar solicitudes ARP por difusión.

- *Ventajas de la caché ARP:* Otorga una respuesta inmediata eliminando las demoras de resolución (*latency*), optimiza el uso de CPU de las estaciones de trabajo y suprime el tráfico de difusión en la infraestructura de conmutación.
- *Problemas asociados a entradas desactualizadas u obsoletas:* Si una entrada permanece en caché más tiempo del debido y las condiciones de la red cambian (por ejemplo, una

máquina cambia de placa de red física, se renueva una concesión DHCP asignando la IP a otro equipo, o un servidor se reemplaza por otro), el emisor continuará enviando las tramas hacia una dirección MAC errónea o inexistente. Las tramas serán ignoradas por el nuevo host o descartadas por la placa receptora, provocando una pérdida total de comunicación hasta que la entrada expire o se purgue manualmente. Además, la ausencia de autenticación intrínseca en ARP expone a la red a ataques de envenenamiento de caché (*ARP spoofing* o *ARP cache poisoning*), donde un host malicioso emite respuestas no solicitadas para asociar fraudulentamente su propia MAC con la IP del gateway o de otros nodos, facilitando ataques de intermediario (*Man-in-the-Middle*).

2.5. Síntesis y evidencias de la práctica

El desarrollo del ensayo permitió comprobar de manera práctica el ciclo completo de resolución de direcciones en redes de área local Ethernet:

- Se inspeccionó la tabla de vecinos en la estación de trabajo Windows, localizando la dirección física del gateway (34-53-d2-98-26-47 para 192.168.0.1) y comprendiendo su función en el encaminamiento de tramas hacia destinos externos.
- Se examinó la estructura del paquete ARP, verificando su encapsulado directo en Ethernet con *EtherType* 0x0806 y la dinámica de solicitud por difusión con *Target MAC* nula frente a respuesta unívoca.
- Se demostró que la transmisión de paquetes de capa de red requiere indispensablemente la previa resolución física en el enlace, explicando por qué una consulta a una IP inexistente genera solicitudes ARP reiteradas pero ningún eco ICMP sobre el medio.

Fuentes técnicas: RFC 826 (*An Ethernet Address Resolution Protocol*); RFC 1122, sección 2.3.2 (*Requirements for Internet Hosts – Communication Layers, Link Layer: ARP*); Manual de usuario de Wireshark.

3. Ítem 3: TCP y UDP “a mano” con ncat

3.1. Fundamentos de TCP, UDP y puertos

a) **Establecimiento de una conexión.** En TCP, establecer una conexión significa que ambos extremos crean y coordinan el estado necesario para intercambiar un flujo de bytes: números de secuencia, confirmaciones, ventanas y opciones. Ese estado reside en las implementaciones TCP de los extremos, normalmente en el núcleo del sistema operativo; no constituye un circuito reservado en los cables. Los routers encaminan datagramas sin necesitar mantener ese estado TCP, aunque dispositivos como cortafuegos o NAT pueden registrar información adicional de los flujos. El intercambio inicial de SYN, SYN/ACK y ACK sincroniza ambos extremos.

b) **Significado de un puerto.** Un puerto es un identificador de 16 bits utilizado por un protocolo de transporte para entregar datos al extremo de comunicación correspondiente. El par (IP, puerto) identifica un extremo dentro de un protocolo dado: TCP y UDP tienen espacios de puertos independientes. Una conexión TCP queda identificada por las direcciones y los puertos de ambos extremos, además del protocolo. Por ello, en loopback las IP pueden ser iguales y los puertos permiten distinguir cliente y servidor.

c) **Proceso en escucha.** Un servidor TCP en escucha posee un socket asociado a una dirección y un puerto, preparado para aceptar conexiones; aún no intercambia datos de aplicación con un cliente concreto. En UDP, “escuchar” significa disponer de un socket asociado a un puerto para recibir datagramas; no existe un establecimiento de conexión equivalente al de TCP. La especificación de TCP se encuentra en el RFC 9293; el formato y servicio de UDP, en el RFC 768.

3.2. Configuración y procedimiento experimental

Se realizaron cinco capturas independientes el 30 de septiembre de 2026, entre las 18:00 y las 18:01 h (UTC−3), con Wireshark 4.2.2 y Ncat 7.94SVN sobre Linux. Ncat se ejecutó desde una extracción temporal del paquete oficial de Ubuntu. La interfaz seleccionada fue `lo`, pues cliente y servidor utilizaron 127.0.0.1. Este tráfico permaneció en el equipo y no circuló por la placa Wi-Fi.

Se comprobó con `ss` que no hubiera servidores escuchando en los puertos 12000 y 12001 antes de iniciar los ensayos. Los comandos del intercambio TCP fueron:

```
# Servidor
ncat -4 -n -v -l 127.0.0.1 12000
# Cliente
ncat -4 -n -v 127.0.0.1 12000
Para UDP se emplearon:
# Servidor
ncat -4 -n -v -u -l 127.0.0.1 12001
# Cliente
ncat -4 -n -v -u 127.0.0.1 12001
```

La opción `-4` fuerza IPv4 y `-n` evita la resolución de nombres. La dirección del servidor se

limitó explícitamente a loopback. En cada protocolo se iniciaron dos procesos Ncat independientes. Sus entradas estándar se alimentaron con tres líneas en cada sentido, separadas aproximadamente 0,4 s, conservando las salidas de ambos procesos. Esta automatización reemplazó la escritura manual de las líneas, pero los sockets y el tráfico fueron generados por Ncat. Se esperó 1,2 s después de iniciar el cliente antes de enviar la primera línea.

Para reproducir **Ctrl+C** se envió **SIGINT** al cliente, con su entrada estándar todavía abierta. Se continuó capturando después de su terminación. El servidor TCP terminó al cerrarse la conexión; el servidor UDP se detuvo posteriormente con la misma señal. Cada captura se inició desde Wireshark y se detuvo automáticamente a los 12 s. Las acciones y sus tiempos quedaron registrados en `capturas/punto3/eventos.json`.

Archivo de captura	Filtro de visualización	Paquetes
<code>tcp_intercambio.pcapng</code>	<code>tcp.port == 12000</code>	18
<code>udp_intercambio.pcapng</code>	<code>udp.port == 12001</code>	6
<code>tcp_una_frase.pcapng</code>	<code>tcp.port == 12000</code>	8
<code>udp_una_frase.pcapng</code>	<code>udp.port == 12001</code>	1
<code>puertos_cerrados.pcapng</code>	<code>tcp.port == 12000 udp.port == 12001 icmp</code>	4

Cuadro 10: Capturas conservadas en `capturas/punto3/`. La numeración de tramas comienza en 1 en cada archivo.

Antes de capturar se aplicaron los filtros de captura `tcp port 12000` o `udp port 12001`. Para puertos cerrados se utilizó `tcp port 12000 or udp port 12001 or (icmp and host 127.0.0.1)`. Estos filtros libpcap limitan la adquisición; los de la tabla se aplican sobre los paquetes ya capturados y utilizan una sintaxis diferente.

En loopback, las dos MAC observadas fueron `00:00:00:00:00:00`. Wireshark utilizó una representación Ethernet II; esas direcciones no corresponden a placas físicas de los extremos. Para visualizar el contenido de Ncat en UDP se seleccionó *Data* mediante *Decode As* para el puerto 12001 (opción `-d udp.port==12001,data`). La decodificación automática por puerto lo interpretaba como LLC; cambiar el decodificador solo corrige su presentación y no modifica los bytes del archivo.

3.3. Mensajes transmitidos y resultados generales

En ambos protocolos se enviaron exactamente las mismas seis líneas. Cada una finalizó con un byte LF (`0x0a`), incluido en las longitudes de la tabla. No se utilizó la conversión a CRLF.

Sentido	Texto, seguido de LF	Bytes	Trama TCP	Trama UDP
Cliente → servidor	Hola desde Ncat	16	4	1
Servidor → cliente	Recibido en el servidor	24	6	2
Cliente → servidor	Segundo mensaje	16	8	3
Servidor → cliente	Segunda respuesta	18	10	4
Cliente → servidor	Fin de la prueba	17	12	5
Servidor → cliente	Prueba completada	18	14	6

Cuadro 11: Correspondencia entre las líneas de Ncat y las tramas de los dos intercambios principales: TCP y UDP.

Las salidas de Ncat confirmaron la recepción de las tres líneas en cada extremo. Se transfirieron 49 bytes desde el cliente y 60 desde el servidor, 109 bytes de aplicación en total. En TCP, el puerto efímero del cliente fue 47154 y el del servidor, 12000; en UDP fueron 51767 y 12001, respectivamente. En ambos sentidos la IP fue 127.0.0.1.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	47154 → 12000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=317954
2	0.000033046	127.0.0.1	127.0.0.1	TCP	74	12000 → 47154 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM T...
3	0.000043377	127.0.0.1	127.0.0.1	TCP	66	47154 → 12000 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3179542487 TSecr=208...
4	1.022640260	127.0.0.1	127.0.0.1	TCP	82	47154 → 12000 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=16 TSval=3179543510 TSe...
5	1.022653575	127.0.0.1	127.0.0.1	TCP	66	12000 → 47154 [ACK] Seq=1 Ack=17 Win=65536 Len=0 TSval=2084034070 TSecr=31...
6	1.423118252	127.0.0.1	127.0.0.1	TCP	90	12000 → 47154 [PSH, ACK] Seq=1 Ack=17 Win=65536 Len=24 TSval=2084034470 TS...
7	1.423142042	127.0.0.1	127.0.0.1	TCP	66	47154 → 12000 [ACK] Seq=17 Ack=25 Win=65536 Len=0 TSval=3179543910 TSecr=2...
8	1.823523727	127.0.0.1	127.0.0.1	TCP	82	47154 → 12000 [PSH, ACK] Seq=17 Ack=25 Win=65536 Len=16 TSval=3179544310 T...
9	1.823552949	127.0.0.1	127.0.0.1	TCP	66	12000 → 47154 [ACK] Seq=25 Ack=33 Win=65536 Len=0 TSval=2084034870 TSecr=3...
10	2.224049124	127.0.0.1	127.0.0.1	TCP	84	12000 → 47154 [PSH, ACK] Seq=25 Ack=33 Win=65536 Len=18 TSval=2084035271 T...
11	2.224066242	127.0.0.1	127.0.0.1	TCP	66	47154 → 12000 [ACK] Seq=33 Ack=43 Win=65536 Len=0 TSval=3179544711 TSecr=2...
12	2.624596043	127.0.0.1	127.0.0.1	TCP	83	47154 → 12000 [PSH, ACK] Seq=33 Ack=43 Win=65536 Len=17 TSval=3179545112 T...
13	2.624610834	127.0.0.1	127.0.0.1	TCP	66	12000 → 47154 [ACK] Seq=43 Ack=50 Win=65536 Len=0 TSval=2084035672 TSecr=3...
14	3.025196769	127.0.0.1	127.0.0.1	TCP	84	12000 → 47154 [PSH, ACK] Seq=43 Ack=50 Win=65536 Len=18 TSval=2084036072 T...
15	3.025211767	127.0.0.1	127.0.0.1	TCP	66	47154 → 12000 [ACK] Seq=50 Ack=61 Win=65536 Len=0 TSval=3179545512 TSecr=2...
16	4.226590094	127.0.0.1	127.0.0.1	TCP	66	47154 → 12000 [FIN, ACK] Seq=50 Ack=61 Win=65536 Len=0 TSval=3179546713 TS...
17	4.226534146	127.0.0.1	127.0.0.1	TCP	66	12000 → 47154 [FIN, ACK] Seq=61 Ack=51 Win=65536 Len=0 TSval=2084037273 TS...

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	UDP	58	51767 → 12001 Len=16
2	0.401060482	127.0.0.1	127.0.0.1	UDP	66	12001 → 51767 Len=24
3	0.802049068	127.0.0.1	127.0.0.1	UDP	58	51767 → 12001 Len=16
4	1.202724358	127.0.0.1	127.0.0.1	UDP	60	12001 → 51767 Len=18
5	1.603353895	127.0.0.1	127.0.0.1	UDP	59	51767 → 12001 Len=17
6	2.003907319	127.0.0.1	127.0.0.1	UDP	60	12001 → 51767 Len=18

Figura 12: Intercambios TCP (arriba) y UDP (abajo). En TCP aparecen segmentos de control además de los datos; en UDP se observan las seis líneas como seis datagramas. Los puertos indican el sentido de cada transferencia.

3.4. a) Tráfico anterior al primer mensaje

En TCP, el arranque del cliente produjo el establecimiento de conexión antes de escribir datos de aplicación. Las primeras tres tramas de `tcp_intercambio.pcapng` fueron:

Trama	Sentido	Flags	Seq	Ack	Datos
1	Cliente → servidor	SYN	0	No válido	0 bytes
2	Servidor → cliente	SYN, ACK	0	1	0 bytes
3	Cliente → servidor	ACK	1	1	0 bytes

Cuadro 12: Establecimiento TCP; Seq y Ack son los valores relativos mostrados por Wireshark. El ACK de la trama 1 no es válido porque el flag ACK vale 0.

El intervalo capturado entre SYN y ACK final fue 43,377 μ s. El primer segmento con datos, trama 4, apareció a los 1,022640260 s respecto del SYN. Así se distingue el establecimiento de transporte del envío del texto. Los SYN también negociaron opciones: MSS=65495, SACK permitido, marcas temporales y escalado de ventana con desplazamiento 10.

En UDP no se capturó ningún paquete de establecimiento. La primera trama contiene directamente **Hola desde Ncat** y LF. El estado local que Ncat configura para recordar el destinatario no implica un intercambio previo con el servidor. Este último pudo responder después del primer datagrama, cuando ya conocía la dirección y el puerto del cliente.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	47154 → 12000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=3179542487
2	0.000023046	127.0.0.1	127.0.0.1	TCP	74	12000 → 47154 [SYN, ACK] Seq=0 Ack=1 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=2
3	0.000043377	127.0.0.1	127.0.0.1	TCP	66	47154 → 12000 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=3179542487 TSecr=2084033847

Figura 13: Las tres tramas de establecimiento TCP, aisladas con un filtro adicional por número de trama. Todavía no contienen bytes de aplicación.

3.5. b) Datagramas por mensaje y confirmaciones

En el ensayo UDP, cada línea introducida produjo un datagrama: tres del cliente y tres del servidor, sin paquetes adicionales de confirmación. Las respuestas textuales del servidor son mensajes de aplicación, no ACK del protocolo UDP. En particular, **Recibido en el servidor** es una línea elegida para el ensayo y no un mecanismo automático de UDP.

En TCP, cada línea produjo en esta ejecución un segmento con datos (PSH, ACK), seguido de un ACK independiente. Por ejemplo, la trama 4 transportó 16 bytes con Seq=1; la trama 5 confirmó Ack=17, es decir, el siguiente byte esperado. Las seis confirmaciones sin datos fueron las tramas 5, 7, 9, 11, 13 y 15. Que aquí exista una correspondencia de una línea por segmento no implica que TCP conserve fronteras de mensajes: la agrupación y segmentación pueden cambiar con el tamaño, el ritmo de escritura y el comportamiento de la pila de red. Tampoco PSH representa un delimitador universal de mensajes.

La comparación del payload capturado y las salidas de Ncat verificó que los textos llegaron completos. En esta prueba no se observaron pérdidas ni retransmisiones TCP. La recepción satisfactoria de seis datagramas locales no demuestra que UDP garantice entrega u orden en otras condiciones.

3.6. c) Comparación de los encabezados

Se comparó la trama TCP 4 con la trama UDP 1 de los intercambios principales. Ambas transportan los mismos 16 bytes:

48 6f 6c 61 20 64 65 73 64 65 20 4e 63 61 74 0a

En ASCII corresponden a `Hola desde Ncat` seguido de LF.

Campo / propiedad	TCP, trama 4	UDP, trama 1
Puerto origen (2 bytes)	47154	51767
Puerto destino (2 bytes)	12000	12001
Secuencia y confirmación	4 bytes cada uno; Seq=1, Ack=1	No existen
Longitud del encabezado	<i>Data Offset</i> =8 palabras: 32 bytes	Fija: 8 bytes
Reservados y flags	Incluidos en el encabezado; PSH y ACK activos	No existen
Ventana (2 bytes)	64; valor escalado 65536 bytes	No existe
Checksum (2 bytes)	0xfe38, <i>unverified</i>	0xfe2b, <i>unverified</i>
Puntero urgente (2 bytes)	0	No existe
Opciones	12 bytes: dos NOP y Timestamp	No existen
Campo <i>Length</i> (2 bytes)	No existe como campo equivalente al de UDP	24 bytes: encabezado y payload
Datos de aplicación	16 bytes	16 bytes

Cuadro 13: Campos observados de transporte. Los tamaños de campos comunes se indican entre paréntesis; Seq/Ack son relativos.

El encabezado TCP mínimo ocupa 20 bytes, pero el segmento analizado ocupa 32 por las opciones. Los SYN de las tramas 1 y 2 tienen encabezados de 40 bytes. Por tanto, no corresponde usar 20 como tamaño de todos los encabezados TCP de esta captura. UDP tiene cuatro campos de 16 bits: puerto origen, puerto destino, longitud y checksum, que suman 8 bytes. El valor *TCP Segment Len*=16 mostrado por Wireshark representa el payload calculado; no es un campo de longitud transmitido en el encabezado TCP.

Los tamaños completos observados fueron:

TCP: 14 (Ethernet) + 20 (IPv4) + 32 (TCP) + 16 (datos) = 82 bytes,

UDP: 14 (Ethernet) + 20 (IPv4) + 8 (UDP) + 16 (datos) = 58 bytes.

Estas son longitudes de captura en loopback, no cantidades transmitidas por un cable. IPv4 identifica TCP mediante *Protocol*=6 y UDP mediante *Protocol*=17.

Los checksums de transporte se mostraron como *unverified*; ese estado indica ausencia de validación, no corrupción demostrada. En las capturas locales también pueden aparecer

checksums parciales por *offloading*, como explica el manual de Wireshark, sección 7.10. No se modificaron los bytes ni la configuración de red para alterar esa presentación.

```

▼ Transmission Control Protocol, Src Port: 47154, Dst Port: 12000, Seq: 1, Ack: 1, Len: 16
  Source Port: 47154
  Destination Port: 12000
  [Stream index: 0]
  ▶ [Conversation completeness: Complete, WITH_DATA (31)]
  [TCP Segment Len: 16]
  Sequence Number: 1      (relative sequence number)
  Sequence Number (raw): 3683170531
  [Next Sequence Number: 17      (relative sequence number)]
  Acknowledgment Number: 1      (relative ack number)
  Acknowledgment number (raw): 1010882309
  1000 .... = Header Length: 32 bytes (8)
  ▶ Flags: 0x018 (PSH, ACK)
  Window: 64
  [Calculated window size: 65536]
  [Window size scaling factor: 1024]
  Checksum: 0xfe38 [unverified]
  [Checksum Status: Unverified]
  Urgent Pointer: 0
  ▶ Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  ▶ [Timestamps]
  ▶ [SEQ/ACK analysis]
  TCP payload (16 bytes)
▶ Data (16 bytes)
▼ User Datagram Protocol, Src Port: 51767, Dst Port: 12001
  Source Port: 51767
  Destination Port: 12001
  Length: 24
  Checksum: 0xfe2b [unverified]
  [Checksum Status: Unverified]
  [Stream index: 0]
  ▶ [Timestamps]
  UDP payload (16 bytes)

```

Figura 14: Detalle de los encabezados: TCP de 32 bytes (arriba), con *Header Length* resaltado; UDP de 8 bytes (abajo), cuyo campo *Length*=24 incluye los 16 bytes de datos.

3.7. d) Cierre del cliente

Al terminar el cliente UDP con SIGINT, no apareció un datagrama de cierre ni una confirmación. La captura conservó únicamente los seis mensajes de aplicación. El servidor no recibió una notificación de fin de sesión del protocolo y permaneció activo hasta su detención explícita.

En TCP, la misma acción produjo un cierre ordenado, no un RST, después de que los datos hubieran sido entregados. Las tramas 16–18 fueron:

Trama	Sentido	Flags	Seq	Ack
16	Cliente → servidor	FIN, ACK	50	61
17	Servidor → cliente	FIN, ACK	61	51
18	Cliente → servidor	ACK	51	62

Cuadro 14: Cierre observado en `tcp_intercambio.pcapng`.

El cliente había transmitido 49 bytes, por lo que su siguiente secuencia relativa era 50. El FIN consume una posición y el servidor lo confirma con Ack=51. De manera análoga,

los 60 bytes del servidor dejan su FIN en Seq=61, confirmado con Ack=62. El servidor combinó la confirmación del FIN recibido con su propio FIN en la trama 17: por ello se observaron *tres* segmentos de cierre, no cuatro segmentos separados. No es una cantidad universal; depende del intercambio y del momento en que cada extremo cierra su sentido de comunicación.

No.	Time	Source	Destination	Protocol	Length	Info
16	4.226508094	127.0.0.1	127.0.0.1	TCP	66	47154 → 12000 [FIN, ACK] Seq=50 Ack=61 Win=65536 Len=0 TSval=3179546713 TSecr=20...
17	4.226534146	127.0.0.1	127.0.0.1	TCP	66	12000 → 47154 [FIN, ACK] Seq=61 Ack=51 Win=65536 Len=0 TSval=2084037273 TSecr=31...
18	4.226549828	127.0.0.1	127.0.0.1	TCP	66	47154 → 12000 [ACK] Seq=51 Ack=62 Win=65536 Len=0 TSval=3179546713 TSecr=2084037...

Figura 15: Las tres tramas del cierre TCP. La trama 17 incorpora simultáneamente FIN y ACK; la 18 confirma el FIN del servidor.

3.8. e) Cantidad de paquetes para enviar la misma frase

Para fijar un criterio de conteo reproducible se realizó un ensayo adicional por protocolo: servidor y cliente nuevos, una única línea *Hola desde Ncat* con LF, ninguna respuesta textual del servidor y cierre posterior del cliente. Se conservaron ambos archivos `tcp_una_frase.pcapng` y `udp_una_frase.pcapng`. El servidor recibió los mismos 16 bytes en los dos casos.

Etapa	TCP	UDP
Establecimiento	3 (tramas 1–3)	0
Envío de la frase	1 (trama 4)	1 (trama 1)
Confirmación sin datos	1 (trama 5)	0
Cierre	3 (tramas 6–8)	0
Total del ensayo	8 paquetes	1 paquete

Cuadro 15: Conteo para una única frase, incluyendo inicio y finalización.

Si solo se cuenta la transferencia sobre una conexión TCP ya establecida, el costo observado fue de dos paquetes (datos y ACK), frente a uno en UDP. El conteo completo de ocho frente a uno incluye el costo de crear y cerrar la conexión. En los intercambios principales de seis líneas, TCP utilizó $3 + 6 + 6 + 3 = 18$ paquetes y UDP, 6. No se registraron duplicados de loopback ni retransmisiones que debieran descontarse.

Los paquetes adicionales de TCP sostienen la entrega ordenada del flujo, las confirmaciones, la recuperación mediante retransmisiones ante pérdidas y el cierre coordinado; junto con el estado de los extremos y los campos del encabezado, también posibilitan control de flujo y de congestión. Un ACK confirma recepción por TCP, no que la aplicación haya procesado semánticamente el mensaje. UDP deja las funciones adicionales de fiabilidad que se necesiten a la aplicación. Esta práctica observa los ACK y la secuencia de control, pero no provoca pérdidas para medir retransmisiones ni congestión.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	44430 → 12000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=526720403 TS...
2	0.000047540	127.0.0.1	127.0.0.1	TCP	74	12000 → 44430 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=3...
3	0.000085785	127.0.0.1	127.0.0.1	TCP	66	44430 → 12000 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=526720404 TSecr=3476512674
4	1.110642534	127.0.0.1	127.0.0.1	TCP	82	44430 → 12000 [PSH, ACK] Seq=1 Ack=1 Win=65536 Len=16 TSval=526721514 TSecr=3476...
5	1.110659004	127.0.0.1	127.0.0.1	TCP	66	12000 → 44430 [ACK] Seq=1 Ack=17 Win=65536 Len=0 TSval=3476513784 TSecr=526721514
6	2.312015600	127.0.0.1	127.0.0.1	TCP	66	44430 → 12000 [FIN, ACK] Seq=17 Ack=1 Win=65536 Len=0 TSval=526722715 TSecr=3476...
7	2.312047987	127.0.0.1	127.0.0.1	TCP	66	12000 → 44430 [FIN, ACK] Seq=1 Ack=18 Win=65536 Len=0 TSval=3476514986 TSecr=526...
8	2.312065490	127.0.0.1	127.0.0.1	TCP	66	44430 → 12000 [ACK] Seq=18 Ack=2 Win=65536 Len=0 TSval=526722716 TSecr=3476514986

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	UDP	58	39735 → 12001 Len=16

Figura 16: Capturas de la frase única: ocho paquetes TCP y un datagrama UDP. El payload de aplicación es idéntico; los paquetes restantes de TCP son de establecimiento, confirmación y cierre.

3.9. f) Comportamiento ante puertos cerrados

Con ambos servidores detenidos se ejecutaron, sucesivamente:

```
ncat -4 -n -v 127.0.0.1 12000
```

```
ncat -4 -n -v -u 127.0.0.1 12001
```

En el segundo proceso se introdujo la línea `Hola` desde Ncat seguida de LF. El archivo `puertos_cerrados.pcapng` registró:

Trama	Protocolo	Puertos / tipo	Interpretación
1	TCP	58404 → 12000	SYN: intento de conexión del cliente.
2	TCP	12000 → 58404	RST, ACK: rechazo del intento; Ack=1 reconoce el SYN.
3	UDP	44168 → 12001	Datagrama con los 16 bytes de la línea.
4	ICMP	Tipo 3, código 3	<i>Destination unreachable / Port unreachable.</i>

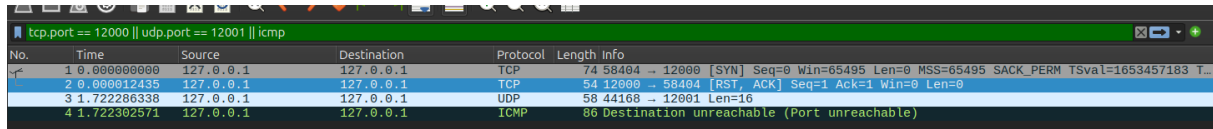
Cuadro 16: Resultados de los intentos sin procesos escuchando. Todas las IP origen y destino son 127.0.0.1.

TCP recibió el RST/ACK aproximadamente $12,435 \mu\text{s}$ después del SYN. Ncat informó **Connection refused** y terminó sin establecer la conexión ni enviar texto. El segmento de rechazo tuvo un encabezado TCP de 20 bytes, sin las opciones negociadas de una conexión establecida.

En UDP, iniciar el cliente no permitió detectar por sí solo el puerto cerrado. Ncat imprimió primero **Connected to 127.0.0.1:12001**; esa indicación describe la asociación local del socket con su destinatario y no acredita respuesta del servidor. Después de enviar el datagrama, el núcleo devolvió el ICMP tipo 3, código 3, unos $16,233 \mu\text{s}$ después de la trama 3. Ncat entonces informó **Connection refused**. Este ICMP es un informe de error de la capa de red, no un ACK de UDP.

La trama ICMP incluyó una copia del datagrama IPv4 que causó el error: en su interior se identifican los puertos UDP 44168 y 12001 y el payload. Así se puede asociar el error con

el envío original. Su longitud capturada fue $14 + 20 + 8 + 44 = 86$ bytes: representación Ethernet, IPv4 exterior, encabezado ICMP y datagrama IPv4 original. La interpretación de tipo y código corresponde al RFC 792. El rechazo explícito observado localmente debe distinguirse de un silencio causado por filtrado o pérdida en una red remota.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	58404 → 12000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1653457183 T...
2	0.000012435	127.0.0.1	127.0.0.1	TCP	54	12000 → 58404 [RST, ACK] Seq=1 Ack=1 Win=0 Len=0
3	1.722286338	127.0.0.1	127.0.0.1	UDP	58	44168 → 12001 Len=16
4	1.722302571	127.0.0.1	127.0.0.1	ICMP	86	Destination unreachable (Port unreachable)

Figura 17: Ensayos a puertos cerrados: SYN seguido de RST/ACK en TCP, y datagrama seguido de ICMP *Port unreachable* en UDP.

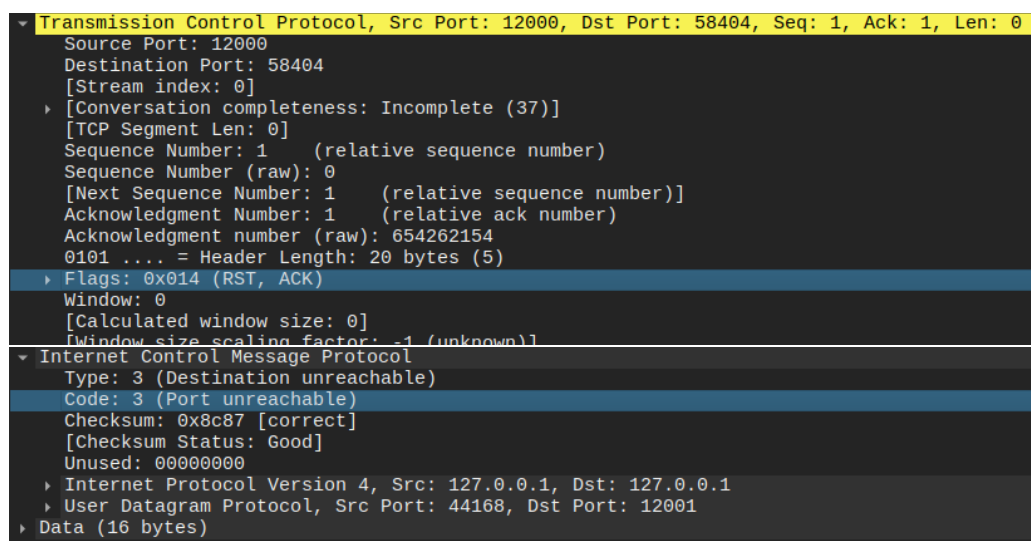


Figura 18: Campos que justifican el rechazo: flags RST y ACK del segmento TCP (arriba) y tipo 3/código 3 del mensaje ICMP (abajo).

3.10. Síntesis y evidencias de la práctica

Los ensayos permitieron distinguir el estado de conexión de TCP del envío de datagramas UDP, comparar sus encabezados y vincular el comportamiento de Ncat con los paquetes capturados. La evidencia muestra tanto el costo de control de TCP como la ausencia de establecimiento y cierre de UDP. Las pruebas sin servidor diferenciaron el rechazo TCP de la notificación ICMP asociada a un envío UDP.

Las cinco capturas y los registros de Ncat se adjuntan en `capturas/punto3/`, junto con los metadatos y las verificaciones. `reproducir.py` permite repetir el ensayo en otra carpeta. Las imágenes originales de Wireshark están en `img/punto3/`; los recortes del informe no alteran sus valores.

Fuentes técnicas: RFC 9293, TCP; RFC 768, UDP; RFC 792, ICMP; Ncat Users' Guide, Basic usage; manual de Wireshark, Checksums. Los números de trama, tamaños y conteos corresponden a las mediciones adjuntas, no a cantidades supuestas a partir de la teoría.

4. Ítem 4: Servidor TCP mínimo - TP5 Redes de Computadoras

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	127.0.0.1	127.0.0.1	TCP	74	35636 → 12000 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=465137352 TSecr=0 WS=1624
2	0.000000000	127.0.0.1	127.0.0.1	TCP	74	12000 → 35636 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=273491042 TSecr=465137352 WS=1624
3	0.000000000	127.0.0.1	127.0.0.1	TCP	82	35636 → 12000 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=465137352 TSecr=273491042
4	0.000000000	127.0.0.1	127.0.0.1	TCP	66	12000 → 35636 [ACK] Seq=1 Ack=17 Win=65536 Len=0 TSval=273491042 TSecr=465137352
5	0.000000000	127.0.0.1	127.0.0.1	TCP	92	12000 → 35636 [PSH, ACK] Seq=1 Ack=17 Win=65536 Len=26 TSval=273491042 TSecr=465137352
6	0.000000000	127.0.0.1	127.0.0.1	TCP	66	35636 → 12000 [ACK] Seq=17 Ack=27 Win=65536 Len=0 TSval=465137352 TSecr=273491042
7	0.000000000	127.0.0.1	127.0.0.1	TCP	66	35636 → 12000 [FIN, ACK] Seq=17 Ack=27 Win=65536 Len=0 TSval=465137352 TSecr=273491042
8	0.000000000	127.0.0.1	127.0.0.1	TCP	66	12000 → 35636 [FIN, ACK] Seq=27 Ack=18 Win=65536 Len=0 TSval=273491042 TSecr=465137352
9	0.000000000	127.0.0.1	127.0.0.1	TCP	66	35636 → 12000 [ACK] Seq=18 Ack=28 Win=65536 Len=0 TSval=465137352 TSecr=273491042

Figura 19: Captura de Wireshark de la conexión TCP entre cliente y servidor.

La captura corresponde a una conexión TCP local sobre la interfaz de loopback: ambos extremos utilizan 127.0.0.1. El cliente emplea el puerto efímero 35636 y el servidor escucha en el puerto 12000. La tabla relaciona las llamadas del programa con los segmentos numerados en Wireshark. Los ACK y el establecimiento de la conexión son gestionados por la pila TCP del sistema operativo.

■ LS python3 tcp_server.py servidor TCP escuchando en 127.0.0.1:12000 ... servidor TCP conexión aceptada desde 127.0.0.1:52248 servidor TCP recibido (16 bytes): 'Agente COMDATOS' servidor TCP respuesta enviada: 'Recibido: Agente COMDATOS' servidor TCP el cliente cerró la conexión ■ LS	■ LS python3 tcp_client.py 127.0.0.1 "Agente COMDATOS" [cliente TCP] conectado a 127.0.0.1:12000 desde 127.0.0.1:52248 [cliente TCP] enviado: 'Agente COMDATOS' [cliente TCP] respuesta (26 bytes): 'Recibido: Agente COMDATOS' ■ LS
---	--

Figura 20: Captura de la consola con la ejecución de los scripts.

La captura muestra la ejecución del servidor y del cliente TCP. Primero se inicia el servidor, que queda a la espera de una conexión en el puerto 12000. Luego, el cliente establece la conexión y envía un mensaje de texto. El servidor recibe los datos y responde anteponiendo **Recibido:** al mensaje recibido.

El cliente muestra la respuesta y cierra la conexión. El servidor detecta este cierre cuando `recv()` devuelve una secuencia de bytes vacía, cierra sus sockets y finaliza. Este intercambio permite verificar el establecimiento de la conexión, la transmisión de datos en ambos sentidos y el cierre de la comunicación.

Llamada	¿Dónde se ejecuta?	¿Genera tráfico?	Segmentos que se observan
<code>socket()</code>	Servidor y cliente	No; operación local.	Ninguno. Crea el socket, sin establecer una conexión.
<code>bind()</code>	Servidor	No; operación local.	Ninguno. Asocia el socket a una IP y un puerto locales.
<code>listen()</code>	Servidor	No por sí misma.	Ninguno al ejecutarse. Deja el socket en escucha; al llegar el SYN, la pila TCP responde con SYN, ACK (segmento 2).
<code>connect()</code>	Cliente	Sí; inicia la conexión.	Three-way handshake: 1 [SYN], cliente a servidor; 2 [SYN, ACK], servidor a cliente; 3 [ACK], cliente a servidor.
<code>accept()</code>	Servidor	No por sí misma.	Ninguno propio. Devuelve un nuevo socket para una conexión establecida; no genera el SYN, ACK.
<code>sendall()</code>	Servidor y cliente	Sí; envía datos.	Cliente: 4 [PSH, ACK], 16 bytes, confirmado por 5 [ACK]. Servidor: 6 [PSH, ACK], 26 bytes, confirmado por 7 [ACK].
<code>recv()</code>	Servidor y cliente	No envía datos por sí misma.	Lee los datos de 4 y 6. Los ACK 5 y 7 los genera TCP, no la llamada. En el servidor, la recepción del FIN de 8 hace que la lectura devuelva bytes vacíos una vez agotados los datos.
<code>close()</code>	Ambos	Sí, al cerrar los sockets conectados.	8 [FIN, ACK], cliente a servidor; 9 [FIN, ACK], servidor a cliente, confirma además el FIN anterior; 10 [ACK], cliente a servidor. Cerrar el socket de escucha no agrega segmentos a esta conexión.

Cuadro 17: Relación entre las llamadas de sockets y los segmentos de la captura TCP.

El cierre observado utiliza tres segmentos porque el servidor combina en el segmento 9 la confirmación del FIN del cliente con su propio FIN. Los 16 bytes enviados por el cliente explican el ACK 17; los 26 bytes enviados por el servidor explican el ACK 27. Cada FIN consume además un número de secuencia, por lo que los valores finales avanzan a 18 y 28, respectivamente.